

The Design and Formal Analysis of the SCB Memory-Hard Function

John G. Underhill

Quantum Resistant Cryptographic Solutions Corporation, Canada

Abstract. SCB is a Keccak based memory hard function designed for key derivation and password hashing in environments where resistance to GPU and ASIC acceleration is required. The construction combines an initialization phase based on cSHAKE, an iterative memory filling procedure that forces latency bound cache thrashing at the granularity of L2 scale strides, and a SHA3 based key evolution step that derives a fresh internal key at each iteration. The final output is generated by a SHAKE extendable output function keyed with the last internal state. The algorithm is fully deterministic and relies on data independent memory access patterns suitable for side channel resistant applications.

This paper presents an implementation-aligned engineering description of SCB and specifies its behavior in a platform independent mathematical form. The dependency structure induced by the scatter based memory traversal is formalized as a directed acyclic graph, and the transcript-reconstruction cost imposed by periodic full-buffer absorption is analyzed in the parallel pebbling model. The analysis shows that honest evaluation uses peak memory $\Theta(M)$, sequential depth $\Omega(C_{\text{cpu}}L)$, and transcript work quadratic in the memory-cost parameter, because the complete buffer is absorbed once per L2-sized region of writes. For memory-reduced evaluators, each full-buffer snapshot must either be stored or reconstructed exactly from the keyed XOF stream and scatter schedule; this creates strong recomputation pressure, although the exact constant-optimal tradeoff curve for all streaming adversaries is left as future work. The security of the construction is examined in the indistinguishability framework for sponge based functions, and the paper provides comparisons with established memory hard functions such as Argon2i, scrypt, Balloon Hashing, Catena, and yescrypt. Performance considerations, implementation guidance, and limitations are discussed in detail.

Keywords: SCB · memory-hard function · Keccak · cSHAKE · SHA3 · SHAKE · password hashing · key derivation · pebbling · side-channel resistance

Contents

1	Introduction	3
1.1	Context and Motivation	3
1.2	Contributions	3
1.3	Relation to Prior Work	4
2	Engineering Description of SCB	4
2.1	Interface, Inputs, and Parameters	4
2.2	Internal State and Keccak Modes	5
2.3	Scatter Pattern and Working Memory	5
2.4	Operational Flow of SCB	6
2.5	Pseudo-code Definition	7
3	Formal Specification of SCB	9
3.1	Notation	9
3.2	Construction as a Memory Hard Function	10
3.3	Scatter Based Memory Fill	10
3.4	Iterative Key Evolution	11
4	Dependency Graph and Memory-Hardness Model	12
4.1	Dependency Graph of SCB	12
4.2	Pebbling Game and Cumulative Memory Complexity	13
4.3	Depth, Transcript Work, and Memory-Reduction Pressure	14
5	Security Model and Sponge Assumptions	15
5.1	Random Oracle and Sponge Indifferentiability	15
5.2	Adversarial Capabilities and Goals	16
5.3	Correctness and Determinism	16
6	Security Analysis	17
6.1	Resistance to Time-Memory Tradeoffs	17
6.2	Precomputation and Pattern Fixity	18
6.3	Side-Channel and Data Independence	19
7	Performance and Implementation Considerations	20
7.1	Complexity and Scaling	20
7.2	Benchmarks	21
7.3	Engineering Guidance	22
8	Comparison with Related Work	23
8.1	Comparison with Argon2 and Balloon Hashing	23
8.2	Comparison with scrypt and yescrypt	24
9	Limitations and Future Work	24
10	Conclusion	26
	References	27

1 Introduction

1.1 Context and Motivation

Memory hard functions are a central component of modern key derivation and password hashing systems. Their purpose is to raise the cost of large scale offline guessing attacks by forcing an adversary to commit a significant amount of memory in addition to computation. This requirement lowers the advantage of massively parallel hardware such as GPUs, FPGAs, and ASICs, which typically offer high arithmetic throughput but have slower access to large pools of memory and limited ability to hide latency when memory access is irregular.

The Password Hashing Competition established rigorous criteria for such constructions. A memory hard function should incur a cost that scales with both memory and time, and it should resist time space tradeoff strategies that attempt to reduce memory at the expense of increased computation. Since the completion of the competition there has been continued interest in new designs that explore alternative approaches to memory hardness. These include bandwidth bound constructions that saturate memory channels and latency bound constructions that attempt to force high cost cache misses.

SCB belongs to the latter category. It is a Keccak based construction that uses cSHAKE for initialization, a scatter based memory fill that forces deterministic cache thrashing, and a SHA3 based key evolution step that binds each iteration to the full memory transcript. The resulting function is designed so that computation is dominated by unpredictable memory access latency rather than arithmetic throughput. This property aims to reduce the advantage of hardware accelerators that can switch threads to mask latency but cannot eliminate the inherent cost of accessing large memory regions with low locality.

1.2 Contributions

This paper provides an implementation-aligned description and analysis of the SCB construction. Its contributions are as follows.

- It defines the SCB algorithm directly from the reference implementation and presents a platform independent engineering description of all internal operations, including initialization, scatter index generation, memory filling, key evolution, and output derivation.
- It introduces a pseudocode specification that formalizes the behavior of the construction and provides a stable interface for analysis and implementation.
- It models the SCB memory filling procedure as a directed acyclic graph and analyzes, in the parallel pebbling model, the sequential depth, honest peak memory, quadratic transcript work, and recomputation pressure induced by periodic full-buffer absorption.
- It analyzes the security of the construction under standard assumptions for sponge based primitives, including indistinguishability from a random oracle and the pseudorandomness of cSHAKE and SHAKE when keyed with high entropy seeds.
- It provides a comparison between SCB and established memory hard functions such as Argon2, Balloon Hashing, scrypt, Catena, and yescrypt, with attention to memory access patterns, side channel properties, and hardware resistance.
- It discusses implementation considerations and performance expectations that should be validated by reproducible benchmark suites on representative CPU and GPU platforms.

1.3 Relation to Prior Work

The design of SCB is related to several established families of memory hard functions. It shares with Argon2i the property of using a deterministic and data independent memory access pattern, which provides resistance to timing and cache based side channel attacks. It differs from the matrix based approach of Argon2 by using a single linear memory region accessed through a scatter pattern intended to maximize cache latency rather than memory bandwidth.

The scatter based memory traversal used in SCB has conceptual similarities to the mixing strategy in Balloon Hashing, which also integrates repeated hashing of memory to increase cumulative cost. However, SCB relies on a structured permutation over cache line indices to induce widespread cache evictions at a fixed stride. This is distinct from the random access patterns used in Balloon Hashing and the data dependent access of scrypt.

Keccak based constructions have previously appeared in Catena and related designs that use sponge functions as the primary mixing primitive. SCB extends this line of work by combining cSHAKE, SHA3, and SHAKE within a unified iterative structure that separates initialization, memory generation, and key evolution. The integration of these components is analyzed in the indifferenciability framework for sponge based hash functions.

Overall, SCB contributes an alternative point in the design space of MHFs by focusing on latency bound behavior driven by deterministic scatter based memory traversal rather than bandwidth bound or data dependent approaches.

2 Engineering Description of SCB

SCB is defined by four public functions and several internal mechanisms that operate on a fixed size Keccak based state. The purpose of this section is to describe the construction in a precise and implementation agnostic manner derived directly from the reference code. The description captures the initialization, memory filling, key evolution, and output generation phases. All behavior is defined in terms of byte strings, Keccak sponge modes, and deterministic memory traversal rules rather than C level operations.

2.1 Interface, Inputs, and Parameters

The SCB interface consists of the following public procedures.

- `qsc_scb_initialize`: initialize a context from seed, info, CPU cost, and memory cost.
- `qsc_scb_generate(ctx, output, outlen)`
- `qsc_scb_update(ctx, seed, seedlen)`
- `qsc_scb_dispose(ctx)`

The input parameters are:

- A seed S of either 32 bytes or 64 bytes. Seed length selects between the 256 bit and 512 bit variants.
- An optional customization string I .
- A CPU cost parameter C_{cpu} in the range 1 to 1000.
- A memory cost parameter C_{mem} in the range 1 to 128, representing mebibytes of RAM.
- An output length ℓ for key derivation.

The internal state `ctx` maintained by SCB consists of:

- An internal key `ckey` of length $k_{\text{len}} \in \{32, 64\}$ bytes.
- A Keccak rate identifier r fixing the security level of every sponge call: the 256-bit security instance (capacity $c = 512$) for 32 byte seeds, or the 512-bit security instance (capacity $c = 1024$) for 64 byte seeds. Throughout this paper we write these two configurations as r_{256} and r_{512} , corresponding to the `qsc_keccak_rate_256` and `qsc_keccak_rate_512` instances of the reference code. The label “512-bit” denotes a 512-bit security XOF realised over Keccak- $f[1600]$ with capacity 1024; it is *not* the FIPS-202 SHAKE256 function and should not be confused with a standardised “SHAKE-512”.
- The stored parameters C_{cpu} and C_{mem} .
- The derived key length k_{len} .

All errors and parameter checks enforced by the implementation are reflected by limiting the above domains. A single rate r is selected at initialization from the seed length and is reused, unchanged, by every cSHAKE, SHAKE, and SHA3 call for the lifetime of the state.

2.2 Internal State and Keccak Modes

SCB uses three modes of the Keccak sponge construction, each applied to a different phase of the algorithm.

- **cSHAKE** is used in initialization to derive the first internal key `ckey` from the seed S , the fixed SCB domain string, and optional customization I .
- **SHA3** is used in each CPU iteration to accumulate a transcript of memory activity. It absorbs the previous key, the sequence of line indices, and periodic full buffer snapshots. Its finalized output becomes the next internal key.
- **SHAKE** is used in memory filling to generate per line blocks and in the final output phase to produce an extendable output of arbitrary length.

The selected rate is determined by the length of the seed and remains constant throughout the entire lifecycle of the state.

2.3 Scatter Pattern and Working Memory

For a memory cost C_{mem} , SCB allocates a contiguous working region of

$$M = C_{\text{mem}} \cdot 2^{20}$$

bytes. This region is divided into cache line sized units of fixed size $\text{CL} = 64$ bytes. The number of logical lines is therefore

$$L = M/\text{CL}.$$

SCB defines a deterministic mapping from logical line indices to physical line indices. The reference routine `scb_scatter_index_dynamic` constructs a permutation π on $\{0, 1, \dots, L - 1\}$ with the following properties.

- The permutation is fixed for a given C_{mem} and does not depend on the seed or the internal key.
- With $\text{lmul} = (L \cdot \text{CL})/\text{L2}$ lanes and $\text{ccnt} = L/\text{lmul}$ lines per lane, two logical indices that are consecutive within a lane map to physical locations exactly $\text{ccnt} \cdot \text{CL} = \text{L2}$

bytes apart, i.e. one L2 stride. The implementation uses a constant stride value of 256 KiB, which may be retargeted by adjusting the macro constant L2.

- Because each successive intra-lane write lands one L2 stride away, it evicts the previously touched line at L2 granularity, forcing a cache miss.

Given a logical index i , the physical write location is

$$p_i = \pi(i), \quad \text{and the byte offset is } p_i \cdot \text{CL}.$$

Remark 1 (π is a genuine permutation on the supported domain). For every admissible memory cost the construction is an exact bijection. With $C_{\text{mem}} \in \{1, \dots, 128\}$ mebibytes and $\text{L2} = 256$ KiB we have $L = C_{\text{mem}} \cdot 2^{14}$, $\text{lmul} = 4 C_{\text{mem}}$, and $\text{ccnt} = 4096$, so $\text{lmul} \cdot \text{ccnt} = L$ exactly and the map $(i, j) \mapsto \text{lmul} \cdot i + j$ ranges bijectively over $\{0, \dots, L - 1\}$ while $(i, j) \mapsto i + j \cdot \text{ccnt}$ ranges over the same set. Hence π is a permutation with no holes or aliasing. The reference code guards the degenerate case $\text{lmul} = 0$ (which arises only for $M < \text{L2}$, impossible for $C_{\text{mem}} \geq 1$) by setting $\text{lmul} = 1$; this branch is therefore never taken on the supported parameter range. If the stride L2 is retargeted, C_{mem} must be chosen so that $\text{L2} \mid M$ and $\text{lmul} \geq 4$, i.e. the working memory is at least four L2 strides; otherwise π degenerates and the cache-miss guarantee is lost.

The permutation table is reconstructed at the start of each memory filling pass, since SCB_FILLMEMORY allocates and computes the scatter indices independently for every CPU iteration.

2.4 Operational Flow of SCB

SCB proceeds in three stages: initialization, iterative key evolution, and final key expansion.

Initialization. The seed S is absorbed into a cSHAKE instance with a fixed SCB domain string and optional customization I . The first squeezed block is taken as the initial internal key ckey , truncated to $k_{\text{len}} = |S|$ bytes. The parameters C_{cpu} and C_{mem} are stored in the state.

Iterative Key Evolution. A working buffer of M bytes is allocated once, cleared before the CPU-cost loop, and reused across all iterations. The scatter permutation is reconstructed within each memory-filling pass. The CPU cost loop is performed for $j = 1$ to C_{cpu} .

In each iteration:

1. A SHA3 state H is initialized.
2. The previous key ckey is absorbed into H .
3. The memory filling procedure SCB_FILLMEMORY is invoked. It uses a SHAKE instance keyed with ckey to generate one block per logical line. Each block is written to its physical location p_i . For each write, index data (i, p_i) is absorbed into H . Periodically, after writing a region equal to the L2 stride, the full buffer is absorbed into H .
4. After all lines are processed, H is finalized to produce the next key ckey .

This produces a chain of internal keys:

$$K_0 \rightarrow K_1 \rightarrow \dots \rightarrow K_{C_{\text{cpu}}}.$$

The working buffer is securely cleared and released after the final iteration.

Remark 2 (Transcript-hash lifecycle). The formal definition of Section 3 models each key-evolution step as an independent hash evaluation $K_j = \text{SHA3}_r(K_{j-1} \parallel T_j)$. The reference implementation initializes the SHA3 transcript state before the CPU-cost loop and finalizes it at the end of every iteration. This is equivalent to per-iteration transcript initialization because the SHA3 finalization routine disposes the Keccak state after emitting the digest, clearing the state array, buffer, and position field. The next update therefore begins from the same zeroed state that explicit reinitialization would produce. The recurrence used below is consequently implementation-aligned.

Output Generation. A SHAKE instance is initialized with the final key $K_{C_{\text{cpu}}}$ and the rate selected at initialization. The requested number of output bytes is obtained by squeezing this instance. This value is the output of SCB.

Update and Disposal. The update function produces a new internal key by hashing the current key together with new seed material through SHA3. Disposal zeroizes the internal key and resets all parameter fields.

2.5 Pseudo-code Definition

The following pseudo-code defines the behavior of the public SCB procedures. Each algorithm is derived directly from the reference implementation and written in an implementation agnostic form. The notation matches the formal specification in the next section. Variables correspond to state fields in the reference code, while mathematical symbols denote their abstract form.

Initialization. The initialization procedure derives the first internal key K_0 from the seed using cSHAKE, after checking that all parameters fall within the supported ranges. It sets the Keccak rate, key length, and cost parameters in the context.

Algorithm 1 SCB_INITIALIZE

Require: Context pointer ctx , seed S , seed length seedlen , optional info string I , CPU cost C_{cpu} , memory cost C_{mem}

Ensure: Internal state ctx initialized, or failure if parameters are invalid

- 1: **if** $\text{ctx} = \perp$ or $S = \perp$ **then**
- 2: **return** failure
- 3: **if** $\text{seedlen} \notin \{32, 64\}$ **then**
- 4: **return** failure
- 5: **if** C_{cpu} not in $[1, 1000]$ or C_{mem} not in $[1, 128]$ **then**
- 6: **return** failure
- 7: $k_{\text{len}} \leftarrow \text{seedlen}$
- 8: **if** $k_{\text{len}} = 32$ **then**
- 9: $\text{ctx.rate} \leftarrow r_{256}$ // 256-bit security, capacity 512
- 10: **else**
- 11: $\text{ctx.rate} \leftarrow r_{512}$ // 512-bit security, capacity 1024
- 12: Initialize cSHAKE with key S , name string "SCB v1.d", customization I
- 13: Squeeze one output block and set ctx.ckey to the first k_{len} bytes
- 14: $\text{ctx.klen} \leftarrow k_{\text{len}}$
- 15: $\text{ctx.cpuc} \leftarrow C_{\text{cpu}}$
- 16: $\text{ctx.memc} \leftarrow C_{\text{mem}}$
- 17: **return** success

Generation. The generation procedure allocates the working memory, performs C_{cpu} iterations of memory fill and key evolution, then expands the final key with SHAKE to produce the output.

Algorithm 2 SCB_GENERATE

Require: Context ctx with initialized key and parameters, desired output length ℓ

Ensure: Output string $Y \in \{0, 1\}^\ell$

```

1:  $M \leftarrow \text{ctx.memc} \cdot 2^{20}$  // Working memory in bytes
2: Allocate buffer  $\text{buf}[0..M - 1]$ 
3: if  $\text{buf} = \perp$  then
4:   return failure
5: for  $j = 1$  to  $\text{ctx.cpuc}$  do
6:   Initialize a fresh SHA3 state  $H$  // see Remark 2
7:   Absorb  $\text{ctx.ckey}$  into  $H$ 
8:   SCB_FILLMEMORY( $\text{ctx}, \text{buf}, M, H$ )
9:   Finalize  $H$  into  $\text{ctx.ckey}$  using rate  $\text{ctx.rate}$  and length  $\text{ctx.klen}$ 
10: Initialize SHAKE instance  $W$  with key  $\text{ctx.ckey}$ , rate  $\text{ctx.rate}$ , and key length  $\text{ctx.klen}$ 
11: Squeeze  $\ell$  bytes from  $W$  to produce  $Y$ 
12: Securely erase and deallocate  $\text{buf}$ 
13: return  $Y$ 

```

Update. The update procedure refreshes the internal key in place by hashing the current key with new seed material.

Algorithm 3 SCB_UPDATE

Require: Context ctx with key ctx.ckey , additional seed S' , seed length seedlen

Ensure: Updated internal key ctx.ckey

```

1: if  $\text{ctx} = \perp$  or  $S' = \perp$  then
2:   return failure
3: Initialize SHA3 state  $H$ 
4: Absorb  $\text{ctx.ckey}$  into  $H$ 
5: Absorb the first  $\text{seedlen}$  bytes of  $S'$  into  $H$ 
6: Finalize  $H$  into  $\text{ctx.ckey}$  using rate  $\text{ctx.rate}$  and length  $\text{ctx.klen}$ 
7: return success

```

Memory fill. The memory filling procedure uses the current key to generate one cache line sized block per logical line and writes it into the buffer according to the scatter permutation. It updates the SHA3 transcript with index information and periodic full buffer snapshots.

Algorithm 4 SCB_FILLMEMORY

Require: Context ctx , buffer $\text{buf}[0..M-1]$, buffer length M , SHA3 state H
Ensure: Buffer filled and transcript H updated

- 1: $\text{CL} \leftarrow 64$
- 2: $L \leftarrow M/\text{CL}$ // Number of cache lines
- 3: $\text{L2} \leftarrow 256 \cdot 2^{10}$ // Assumed L2 size in bytes
- 4: $\text{LPR} \leftarrow \text{L2}/\text{CL}$ // Lines per L2 sized region = 4096
- 5: Allocate integer array $\pi[0..L-1]$
- 6: $\text{SCB_SCATTERINDEXDYNAMIC}(\pi, L, \text{CL}, \text{L2})$
- 7: Initialize SHAKE instance W with key ctx.ckey , rate ctx.rate , length ctx.klen
- 8: **for** $i = 0$ to $L-1$ **do**
- 9: $p \leftarrow \pi[i]$
- 10: $B \leftarrow \text{SHAKE_Squeeze}(W, \text{CL})$ // Next cache line block
- 11: Write B into $\text{buf}[p \cdot \text{CL}..p \cdot \text{CL} + \text{CL} - 1]$
- 12: $\text{SHA3_Absorb}(H, \text{enc}_{64}(i) \parallel \text{enc}_{64}(p))$
- 13: **if** $(i+1) \bmod \text{LPR} = 0$ **then**
- 14: $\text{SHA3_Absorb}(H, \text{buf})$ // absorb the entire M -byte buffer
- 15: Securely erase and deallocate π

Scatter index construction. The scatter index construction defines the permutation of cache line indices that causes each successive logical index to land in a different L2 sized lane of memory.

Algorithm 5 SCB_SCATTERINDEXDYNAMIC

Require: Integer array $\pi[0..L-1]$, number of lines L , cache line size CL , L2 size L2
Ensure: π filled with a permutation of $\{0, \dots, L-1\}$

- 1: $\text{lmul} \leftarrow (L \cdot \text{CL})/\text{L2}$ // Number of lanes
- 2: $\text{ccnt} \leftarrow L/\text{lmul}$ // Lines per lane
- 3: **for** $i = 0$ to $\text{ccnt} - 1$ **do**
- 4: **for** $j = 0$ to $\text{lmul} - 1$ **do**
- 5: $\pi[(\text{lmul} \cdot i) + j] \leftarrow i + (j \cdot \text{ccnt})$

3 Formal Specification of SCB

This section defines SCB as a deterministic memory hard function over bitstrings. The definition follows the engineering description in the previous section but is expressed in mathematical terms suitable for formal analysis, security proofs, and graph based modeling.

3.1 Notation

We use the following general notation.

- $\{0,1\}^n$ denotes the set of bitstrings of length n . The set $\{0,1\}^*$ denotes variable length bitstrings.
- Concatenation of bitstrings X and Y is written $X \parallel Y$.
- For a byte array A and integer index i , the symbol $A[i]$ denotes the i -th byte. A slice $A[a..b]$ denotes the inclusive range $A[a], A[a+1], \dots, A[b]$.
- For an integer x , the function $\text{enc}_{64}(x)$ denotes a fixed length 64 bit encoding of x .

We use the following Keccak based primitives.

- $\text{cSHAKE}_r(S, N, I)$ is the customizable SHAKE function with rate r , keyed with bitstring S , using name string N and customization string I .
- $\text{SHAKE}_r(K)$ denotes an extendable output function with rate r keyed by K . The function $\text{SHAKE}_r(K, \ell)$ returns ℓ output bytes.
- $\text{SHA3}_r(X)$ denotes the fixed output hash function with rate r . Hashing is described by the absorb then finalize operations of the Keccak sponge.

We fix the following SCB parameters.

- $k \in \{256, 512\}$ denotes the key size in bits. The derived internal key has length $k_{\text{len}} = k/8$ bytes.
- $C_{\text{cpu}} \in \{1, \dots, 1000\}$ is the number of CPU iterations.
- $C_{\text{mem}} \in \{1, \dots, 128\}$ determines the working memory size $M = C_{\text{mem}} \cdot 2^{20}$ bytes.
- The buffer is divided into $L = M/\text{CL}$ lines, where $\text{CL} = 64$ is the cache line size.
- The implementation uses a fixed cache stride value $\text{L2} = 256 \cdot 2^{10}$.

All operations are deterministic and the function outputs depend only on the inputs $(S, I, C_{\text{cpu}}, C_{\text{mem}}, \ell)$.

3.2 Construction as a Memory Hard Function

We define the SCB function as

$$\text{SCB}(S, I, C_{\text{cpu}}, C_{\text{mem}}, \ell) \in \{0, 1\}^\ell,$$

where S is a seed of length 32 or 64 bytes, I is an optional customization string, and ℓ is the desired output length.

The construction proceeds through the following phases.

Initialization.

$$K_0 = \text{cSHAKE}_r(S, N_{\text{SCB}}, I)[0..k_{\text{len}} - 1],$$

where N_{SCB} is the fixed domain string used by SCB.

Iteration. For $j = 1$ to C_{cpu} the next key is defined as

$$K_j = \text{SHA3}_r(K_{j-1} \parallel T_j),$$

where T_j is the memory transcript produced by the memory filling procedure in iteration j . The transcript encodes the sequence of line indices and buffer states generated during that iteration.

Output. The final result is obtained by

$$Y = \text{SHAKE}_r(K_{C_{\text{cpu}}}, \ell).$$

The value Y is the output of the SCB function.

3.3 Scatter Based Memory Fill

The memory filling stage is defined formally as follows.

Scatter Permutation. Let $L = M/\text{CL}$ denote the number of logical cache line blocks. SCB defines a deterministic permutation

$$\pi : \{0, 1, \dots, L-1\} \rightarrow \{0, 1, \dots, L-1\}$$

constructed as a function of L and the fixed stride parameter L2 . The permutation is defined so that consecutive logical indices map to physical locations separated by approximately L2 bytes. This ensures low locality and high latency during memory traversal.

Block Generation. A working buffer buf of length M is allocated once before the iteration loop and initialized to zero. For each iteration j , a keyed XOF instance is initialized with key K_{j-1} :

$$W_j = \text{SHAKE}_r(K_{j-1}).$$

For each logical index $i \in \{0, 1, \dots, L-1\}$, SCB defines the block

$$B_{j,i} = W_j(i),$$

where $W_j(i)$ denotes the next CL bytes squeezed from the SHAKE instance. The block is written to physical offset $\pi(i) \cdot \text{CL}$ in the buffer.

Transcript Construction. Let $\text{LPR} = \text{L2}/\text{CL} = 4096$ and let

$$Q = \{q \cdot \text{LPR} - 1 : 1 \leq q \leq L/\text{LPR}\}$$

be the set of logical indices after which a full-buffer snapshot is taken. For $q \in Q$ write $\text{buf}^{(q)} \in \{0, 1\}^{8M}$ for the entire contents of the working buffer immediately after index q has been written. The memory transcript for iteration j is

$$T_j = \left(\left\|_{i=0}^{L-1} (\text{enc64}(i) \parallel \text{enc64}(\pi(i))) \right\| \parallel \left(\left\|_{q \in Q} \text{buf}^{(q)} \right\| \right) \right).$$

The first group encodes the content-independent sequence of logical and physical line indices $(i, \pi(i))$. The second group encodes periodic snapshots of the entire persistent working buffer. During iteration $j > 1$, early snapshots may still contain lines last written in iteration $j-1$ until the current scatter pass overwrites them; this is an implementation property of the reused buffer and is included in the transcript definition. Because L is an exact multiple of LPR for every admissible C_{mem} (Section 2.3), the largest element of Q is $L-1$, so the final snapshot $\text{buf}^{(L-1)}$ is taken after all L current-iteration blocks have been written. The transcript therefore commits to the complete buffer state at multiple points and, in particular, to every block generated in the current iteration. This full-buffer absorption, rather than the content-independent index stream, is what creates the transcript-reconstruction cost analyzed in Section 4.

3.4 Iterative Key Evolution

Given the transcript T_j of the j -th iteration, the next key is computed by applying SHA3 at rate r to the concatenation:

$$X_j = K_{j-1} \parallel T_j.$$

Formally,

$$K_j = \text{SHA3}_r(X_j)[0..k_{\text{len}}-1].$$

Here SHA3_r denotes the fixed-output Keccak hash at the rate r selected in Section 2.1, whose native digest length is exactly $k_{\text{len}} \in \{32, 64\}$ bytes (a 256-bit digest for r_{256} and a

512-bit digest for r_{512}). The truncation to k_{len} bytes is therefore the identity in the reference implementation and is written explicitly only to make the key length unambiguous in the formal definition.

This produces a sequence of internal keys:

$$K_0, K_1, \dots, K_{C_{\text{cpu}}}$$

that are fully determined by the initial seed, the parameters, and the memory structure. The final output is produced by the SHAKE expansion defined earlier.

4 Dependency Graph and Memory-Hardness Model

This section formalizes the structure of SCB as a directed acyclic graph. The graph captures how each block in the working memory and each intermediate key depends on previous computation. This representation is used to examine memory hardness and resistance to time space tradeoff strategies. Our treatment follows the general approach used in pebbling based analyses of memory hard functions.

4.1 Dependency Graph of SCB

The execution of SCB for parameters $(C_{\text{cpu}}, C_{\text{mem}})$ induces a directed acyclic graph

$$G = (V, E).$$

The node set V and edge set E are defined as follows.

Nodes. The node set is the disjoint union of:

- Key nodes $K_0, K_1, \dots, K_{C_{\text{cpu}}}$.
- Memory block nodes $B_{j,i}$ for each iteration $j \in \{1, \dots, C_{\text{cpu}}\}$ and each logical index $i \in \{0, \dots, L-1\}$.

Each $B_{j,i}$ corresponds to the CL byte block generated at iteration j and logical index i .

Edges. An edge $(u, v) \in E$ means that the value of node v cannot be computed without first computing u .

The edges of G are defined by the following rules.

1. **Key to block dependency.** Each block $B_{j,i}$ depends on K_{j-1} because the SHAKE generator used to produce the block is keyed with K_{j-1} :

$$(K_{j-1}, B_{j,i}) \in E.$$

2. **Block order dependency.** The blocks generated by a SHAKE instance at iteration j form a sequential chain of dependencies:

$$(B_{j,i}, B_{j,i+1}) \in E \quad \text{for all } 0 \leq i < L-1.$$

This reflects that SHAKE produces blocks sequentially and that $B_{j,i}$ precedes $B_{j,i+1}$ in the output stream.

3. **Transcript dependency.** The next key K_j depends on the streaming transcript state that absorbs index pairs and periodic full-buffer snapshots. Since the final snapshot in each pass is taken after all L current-iteration blocks have been written, $K_j = \text{SHA3}_r(K_{j-1} \parallel T_j)$ is a function of every block $B_{j,i}$ generated in that iteration. In the graph abstraction we record this dependence by edges from each block to the transcript or snapshot node that contains it and from the final transcript node to K_j . Equivalently, for reachability arguments one may contract the transcript nodes and write $(B_{j,i}, K_j) \in E$ for every i ; this contraction records functional dependence but does not assert that all blocks must be resident simultaneously in a streaming implementation.

These relations induce a layered structure:

$$K_0 \rightarrow \{B_{1,i}\}_{i=0}^{L-1} \rightarrow K_1 \rightarrow \{B_{2,i}\}_{i=0}^{L-1} \rightarrow \cdots \rightarrow K_{C_{\text{cpu}}}.$$

The graph is acyclic by construction since dependencies flow forward by iteration index and within iteration from lower index blocks to higher index blocks.

4.2 Pebbling Game and Cumulative Memory Complexity

To evaluate resistance to time space tradeoff attacks, we analyze SCB under the parallel pebbling model. This model abstracts computation as placing and removing pebbles on nodes of G , subject to the rule that a node can be pebbled only if all its parents are pebbled.

A pebbling strategy describes how an adversary computes the final output while possibly attempting to save memory by discarding blocks and recomputing them later. The cost of a strategy is measured as cumulative memory complexity (CMC):

$$\text{CMC}(G) = \sum_{t=1}^T M_t,$$

where M_t is the number of pebbles on the graph at time t and T is the total number of computational steps.

The adversary attempts to minimize $\text{CMC}(G)$ under the constraint that it must eventually compute $K_{C_{\text{cpu}}}$ and produce its descendants.

In the SCB graph, the following structural features affect the pebbling cost.

- The length L of each layer is proportional to C_{mem} .
- The number of layers is C_{cpu} .
- Each layer contains a long chain $B_{j,0} \rightarrow B_{j,1} \rightarrow \cdots \rightarrow B_{j,L-1}$ that must be produced sequentially.
- The computation of K_j depends on the full-buffer transcript snapshots, so every block value must either be available at snapshot time or be reconstructed exactly.

A straightforward evaluator stores the working buffer, writes every generated line to its scatter location, and absorbs periodic full-buffer snapshots directly from memory. Its peak working memory is therefore $\Theta(L)$ cache-line blocks, and its transcript work is dominated by the repeated absorption of the complete M -byte buffer. A memory-reduced evaluator can attempt to discard buffer lines and reconstruct them at snapshot time, but every omitted line in a snapshot must then be regenerated from the sequential keyed XOF stream and scatter schedule. Section 4.3 formalizes the resulting depth and honest-work profile and records the recomputation obligation for memory-reduced strategies. The exact constant-optimal tradeoff curve for all streaming adversaries is not claimed here.

4.3 Depth, Transcript Work, and Memory-Reduction Pressure

We now make the graph-based observations precise. The analysis is stated in a parallel pebbling style, but with an important qualification: SCB uses streaming SHA3 absorption for full-buffer snapshots. A contracted edge from every block to K_j records functional dependence, not a simultaneous-residence requirement. A memory-reduced evaluator may therefore recompute and stream snapshot contents instead of storing the entire layer. The results below establish the sequential depth, honest memory and work profile, and the reconstruction obligation faced by such evaluators; they do not claim a tight lower bound for every possible streaming recomputation strategy.

Theorem 1 (Sequential depth). *The longest directed generation path in the SCB dependency graph has length at least $C_{\text{cpu}} \cdot L$. Hence, even with unbounded parallelism outside the keyed XOF stream, any evaluation of SCB has sequential depth $\Omega(C_{\text{cpu}} \cdot L)$ block-generation steps.*

Proof. Within iteration j , the XOF stream generates $B_{j,0}, B_{j,1}, \dots, B_{j,L-1}$ sequentially from K_{j-1} . Computing $B_{j,i+1}$ requires advancing the same XOF stream beyond $B_{j,i}$, giving a directed generation chain through all L logical blocks. The output of iteration j is the key K_j , which is derived from a transcript that includes the final full-buffer snapshot after all L current blocks have been written. The key K_j then seeds the next iteration. Concatenating these chains over C_{cpu} iterations gives a directed path of length at least $C_{\text{cpu}}L$. $\square$

Proposition 1 (Honest evaluation profile). *The straightforward evaluator uses $\Theta(M)$ bytes of working memory and performs $\Theta(C_{\text{cpu}}C_{\text{mem}}^2)$ bytes of full-buffer transcript absorption. Equivalently, with $L = M/64$, the dominant snapshot work is $\Theta(C_{\text{cpu}}L^2)$ up to the fixed cache-line and stride constants.*

Proof. The implementation allocates one $M = C_{\text{mem}}2^{20}$ -byte buffer and reuses it throughout the CPU-cost loop, so the honest peak working memory is $\Theta(M)$. Each iteration writes $L = M/64$ cache-line blocks and absorbs the entire M -byte buffer once every $\text{LPR} = 4096$ writes. Since $L/\text{LPR} = 4C_{\text{mem}}$ for the supported parameters, the full-buffer absorption volume per iteration is

$$4C_{\text{mem}} \cdot M = 4C_{\text{mem}} \cdot C_{\text{mem}}2^{20} = C_{\text{mem}}^22^{22} \text{ bytes.}$$

Multiplying by C_{cpu} gives $\Theta(C_{\text{cpu}}C_{\text{mem}}^2)$ bytes absorbed. Since $L = C_{\text{mem}}2^{14}$, this is also $\Theta(C_{\text{cpu}}L^2)$ after absorbing fixed constants into the asymptotic notation. $\square$

Proposition 2 (Snapshot reconstruction obligation). *For every full-buffer snapshot absorbed during iteration j , an evaluator must provide the exact M -byte buffer state at that snapshot point. Any line not retained in memory must be recomputed from K_{j-1} by replaying the keyed XOF stream to the corresponding logical block and applying the scatter permutation. Consequently, reducing storage below the full working buffer converts saved memory into sequential replay work; it does not remove the transcript obligation.*

Proof. The SHA3 transcript input includes the complete buffer contents at each snapshot point. A correct evaluator must therefore feed SHA3 the same byte string as the reference evaluation. The contents of a line at a snapshot are either retained from the current working buffer, inherited from the previous iteration until overwritten, or generated during the current iteration by the keyed XOF stream and scatter write. If the evaluator discards a required line, the only way to reconstruct it without guessing is to reproduce the deterministic computation that generated the line, which requires advancing the relevant keyed XOF stream in sequence and applying the public scatter map. Thus memory reduction replaces storage with replay and recomputation of transcript material. $\square$

Corollary 1 (Tradeoff interpretation). *SCB imposes a quadratic honest-work profile in the memory-cost parameter and strong recomputation pressure on memory-reduced evaluators. The present analysis does not claim that peak space below M bytes is impossible in an abstract streaming model, nor does it give a constant-optimal tradeoff curve for all budgets. It establishes the implementation-aligned depth, honest memory, and transcript-work costs, and identifies the precise mechanism by which omitted buffer state must be reconstructed.*

The analysis therefore supports SCB as a latency oriented and data independent MHF whose cost is driven by sequential XOF generation, scatter writes, and repeated full-buffer transcript absorption. A tighter adversarial time-memory tradeoff theorem, including exact constants and models that explicitly charge both Keccak permutation work and memory-resident state, remains a natural subject for future analysis.

5 Security Model and Sponge Assumptions

This section formalizes the assumptions used in the analysis of SCB. The construction relies on Keccak based primitives and is analyzed under models commonly used in the study of hash functions and memory hard functions. We describe the random oracle view of cSHAKE, SHAKE, and SHA3, define the adversarial capabilities considered in the SCB threat model, and state correctness and determinism properties of the construction.

5.1 Random Oracle and Sponge Indifferentiability

SCB depends on the cryptographic properties of cSHAKE, SHAKE, and SHA3. All three are instances of the Keccak sponge construction. Their security is analyzed under the indifferentiability framework of Maurer, Renner, and Holenstein.

Sponge Indifferentiability. The sponge construction with a random permutation has been shown to be indifferentiable from a random oracle provided that:

- the rate is large enough relative to the desired output size,
- the adversary is limited to classical queries,
- the capacity matches the intended security level.

For Keccak based primitives with capacity c , the resulting random oracle bound holds up to approximately $2^{c/2}$ queries. SCB selects one of two configurations at initialization (Section 2.1): the r_{256} instance has capacity $c = 512$, giving a generic indifferentiability bound of about 2^{256} queries; the r_{512} instance has capacity $c = 1024$, giving about 2^{512} queries. In both cases the capacity is set so that the sponge bound meets or exceeds the 256-bit or 512-bit security target associated with the seed length, and the rate is large enough that the truncation of Section 3.4 returns the full native digest.

Assumptions for SCB. We adopt the standard assumptions used in prior analyses of cSHAKE and SHAKE:

- $\text{cSHAKE}_r(S, N, I)$ is modeled as a pseudo-random function keyed by S when the name string N and customization I are fixed. This assumption is justified by the indifferentiability of cSHAKE from a random oracle.
- $\text{SHAKE}_r(K)$ is treated as a random oracle keyed by the bitstring K for the purpose of generating memory blocks. This model captures the sequential and unpredictable nature of block generation.

- $\text{SHA3}_r(X)$ is modeled as a random oracle for analyzing the evolution of the internal key K_j from the transcript T_j . Since SCB uses fixed output SHA3 only for intermediate key derivation, this model is consistent with the standard usage of SHA3 in keyed or domain separated modes.

These assumptions allow us to treat all outputs of Keccak based primitives in SCB as uniformly random conditioned on their inputs, while maintaining the structural constraints imposed by the sponge construction.

5.2 Adversarial Capabilities and Goals

We consider a computationally bounded adversary $\mathcal{A}$ with the following capabilities, consistent with standard practice in the analysis of memory hard functions.

Offline Guessing. The adversary may attempt offline guessing attacks against user supplied passwords or low entropy seeds. For each candidate guess S' , the adversary computes $\text{SCB}(S', I, C_{\text{cpu}}, C_{\text{mem}}, \ell)$. Security is achieved when the amortized cost of producing each output is high relative to the cost of a single online use by an honest user.

Time Space Tradeoff Attacks. The adversary may reduce the amount of available memory and attempt to compensate by recomputing blocks of the SCB working memory on demand. The cost of such attacks is evaluated using the pebbling model over the dependency graph defined in the previous section. The goal of the adversary is to minimize cumulative memory complexity while still recovering the final key $K_{C_{\text{cpu}}}$.

Precomputation and Amortization. SCB uses a deterministic scatter pattern that is independent of the seed. An adversary may attempt to exploit this property by precomputing a partial representation of the scatter traversal or by hardwiring the memory schedule into an ASIC. However, since the SHAKE based block generation depends on the unknown internal key K_{j-1} , precomputation of block contents is not possible and the transcript structure necessarily depends on every preceding block.

We assume the adversary may compute multiple instances of the SCB graph in parallel but cannot share intermediate values between unrelated seeds without recomputing all dependent nodes.

Oracle Access. The adversary may query the Keccak based primitives as black box oracles as permitted by the model but has no access to the internal state of the construction and cannot influence the scatter permutation.

These capabilities define the space of feasible adversarial strategies under which the security of SCB is evaluated.

5.3 Correctness and Determinism

SCB is a deterministic function of its inputs. Correctness follows from the fact that all internal steps are uniquely determined by the seed, customization string, and cost parameters. We record the following lemma for completeness.

Lemma 1 (Determinism). *For fixed inputs $(S, I, C_{\text{cpu}}, C_{\text{mem}}, \ell)$ the function SCB always produces the same output. Moreover, for any two different seeds $S \neq S'$ the probability (over the random oracle model for Keccak based functions) that $\text{SCB}(S, I, C_{\text{cpu}}, C_{\text{mem}}, \ell) = \text{SCB}(S', I, C_{\text{cpu}}, C_{\text{mem}}, \ell)$ is negligible.*

Proof. Determinism is immediate: every phase of SCB — the initial cSHAKE key derivation, the keyed SHAKE block generation, the deterministic scatter permutation π , the transcript assembly, and each SHA3 key-evolution step — is a deterministic function once $(S, I, C_{\text{cpu}}, C_{\text{mem}}, \ell)$ are fixed, so repeated evaluation yields identical output.

For the uniqueness bound, model cSHAKE as a random oracle. The initial key is $K_0 = \text{cSHAKE}_r(S, N, I)$ truncated to $k_{\text{len}} = 8 \text{ klen}$ bits, where $\text{klen} \in \{32, 64\}$ bytes. For distinct seeds $S \neq S'$ the inputs to the oracle differ, so the outputs K_0, K'_0 are independent uniform k_{len} -bit strings; thus $\Pr[K_0 = K'_0] = 2^{-8 \text{ klen}}$. Conditioned on $K_0 \neq K'_0$, the two executions drive independent SHAKE and SHA3 oracle chains, and the final outputs collide only if every one of the C_{cpu} key-evolution steps happens to re-synchronise the two chains or the final SHAKE expansion collides on ℓ bytes. A union bound over these events gives

$$\Pr_{S \neq S'}[\text{SCB}(S, \cdot) = \text{SCB}(S', \cdot)] \leq 2^{-8 \text{ klen}} + C_{\text{cpu}} \cdot 2^{-8 \text{ klen}} + 2^{-8\ell},$$

where the first term bounds an initial-key collision, the middle term bounds a re-synchronisation at any of the C_{cpu} SHA3 steps, and the last bounds a collision in the final ℓ -byte expansion. Since $\text{klen} \geq 32$ and $C_{\text{cpu}} \leq 1000$, this probability is at most $(C_{\text{cpu}} + 1)2^{-256} + 2^{-8\ell}$, which is negligible. $\square$

We also record the one-wayness and pseudorandomness of the output, which follow from the same oracle model.

Proposition 3 (PRF output and preimage resistance). *Model cSHAKE, SHAKE, and SHA3 as random oracles. For a uniformly random secret seed S and fixed $(I, C_{\text{cpu}}, C_{\text{mem}}, \ell)$, the output $\text{SCB}(S, I, C_{\text{cpu}}, C_{\text{mem}}, \ell)$ is indistinguishable from a uniform ℓ -byte string by any adversary making q oracle queries, with advantage at most $q \cdot 2^{-8 \text{ klen}}$. Moreover, given only the output, recovering S (or any internal key K_j) requires $\Omega(2^{8 \text{ klen}})$ oracle queries in expectation.*

Proof. The output is $\text{SHAKE}_r(K_{C_{\text{cpu}}})$ truncated to ℓ bytes, and $K_{C_{\text{cpu}}}$ is the result of a chain of random-oracle evaluations seeded by $K_0 = \text{cSHAKE}_r(S, N, I)$. An adversary distinguishes the output from uniform only if it queries some oracle on the exact internal value $K_{C_{\text{cpu}}}$ (or an earlier K_j on the unique path to it); each of its q queries hits a fixed unknown k_{len} -bit value with probability $2^{-8 \text{ klen}}$, giving advantage at most $q \cdot 2^{-8 \text{ klen}}$ by a union bound. The same argument shows that inverting the output to recover any K_j , and hence S , requires guessing a k_{len} -byte random-oracle input, which needs $\Omega(2^{8 \text{ klen}})$ queries in expectation. $\square$

This completes the security model used for the analysis of SCB.

6 Security Analysis

This section analyzes the security properties of SCB under the assumptions and adversarial model described earlier. We focus on three aspects: resistance to time memory tradeoff strategies, resistance to precomputation attacks despite the use of a fixed scatter pattern, and resistance to timing and cache based side channels due to data independent memory access.

6.1 Resistance to Time-Memory Tradeoffs

Time memory tradeoff attacks attempt to reduce memory usage while compensating for the loss by performing additional computation. The dependency graph model developed above captures the sequential generation and transcript obligations imposed by SCB. The

exact constant-optimal tradeoff curve for all streaming recomputation strategies remains open, so the analysis below states only the claims supported by the implementation-aligned graph.

Sequential dependency within iterations. For each iteration j the blocks $B_{j,i}$ form a sequence that must be generated in order. Since the SHAKE stream is sequential, computing $B_{j,i}$ requires computing all blocks $B_{j,t}$ for $t < i$. This yields a linear chain of dependencies of length L .

Any adversary that discards block $B_{j,i}$ must recompute the entire prefix $\{B_{j,0}, \dots, B_{j,i}\}$ in order to regenerate it. This recomputation cost grows linearly with i .

Full block dependency for key derivation. The value of K_j depends on the transcript T_j , and T_j incorporates every block $B_{j,i}$ through index absorption and periodic full buffer absorption. Therefore an adversary must produce every block in order to compute K_j .

This prevents the adversary from computing a subset of blocks and skipping the remainder since the missing blocks would leave unknown components in the transcript and affect the resulting key.

Cumulative cost across iterations. The dependency graph spans C_{cpu} iterations. Key K_j depends on all blocks of iteration j , and each block of iteration j depends on the entire chain of blocks within iteration j . Thus the adversary faces a repeated cost structure across all j .

The cost of skipping memory in iteration j is not amortized across iterations since the block generation for iteration $j + 1$ depends only on K_j , not on the memory contents. This means that an adversary must incur the recomputation cost independently in each iteration.

Tradeoff behavior. Let m be the amount of working storage, measured in cache-line blocks, that an evaluator retains during an iteration. If the evaluator stores the complete buffer, it can absorb every full-buffer snapshot directly and follows the honest profile of Proposition 1. If it stores fewer than L lines, it may still complete the computation in an abstract streaming model, but every missing line required by a snapshot must be reconstructed exactly. By Proposition 2, this reconstruction requires replaying the keyed XOF stream and scatter schedule for omitted lines, and repeated snapshots can force repeated replay of the same prefixes.

The paper therefore does not claim that sublinear-space evaluation is impossible. The defensible conclusion is narrower and implementation-aligned: reducing memory below the full buffer does not eliminate the full-buffer transcript obligation; it shifts cost into sequential recomputation. Quantifying the exact optimal tradeoff for all budgets $m < L$, including adversaries that interleave storage, replay, and streaming SHA3 absorption, is left for future work.

Summary. The structure of SCB forces an adversary to allocate memory proportional to L ; there is no sub-quadratic alternative, because peak space below L is impossible and any feasible strategy already incurs cumulative cost $\Theta(C_{\text{cpu}} L^2)$. This property is consistent with the goals of memory hard function design and provides strong resistance to time memory tradeoff attacks.

6.2 Precomputation and Pattern Fixity

SCB uses a deterministic scatter pattern for memory traversal. An adversary may therefore attempt to exploit the fact that the physical addresses written during the memory fill

phase are known in advance. We argue that this does not allow useful precomputation.

Block contents depend on the internal key. Even though the physical index permutation is fixed, the content of each block $B_{j,i}$ is determined by the SHAKE stream keyed with K_{j-1} . Since the adversary does not know K_{j-1} prior to computation, it cannot precompute or compress block contents.

Transcript coupling prevents decoupled computation. The transcript T_j incorporates index values and full buffer snapshots. These values depend on the actual contents of the buffer after each write and therefore on the evolving SHAKE output. As a result, the adversary cannot fabricate or guess the transcript structure without generating all blocks exactly as SCB does.

Fixed permutation does not reduce memory requirements. The adversary does not gain an advantage by knowing π in advance since the cost of generating blocks is dominated by the sequential nature of the SHAKE stream and the dependency of K_j on all block outputs. Even with prior knowledge of π , the adversary cannot shortcut the generation of blocks or reduce the dependency graph.

ASIC considerations. While the scatter pattern could in principle be embedded into custom hardware, the primary cost of SCB comes from memory latency. Embedding the permutation into hardware does not change the fact that blocks must be generated sequentially from the SHAKE stream and that buffer updates must occur across the full memory region. Therefore custom hardware must still provision either sufficient memory bandwidth and capacity for the snapshots or additional recomputation logic; the exact advantage is architecture-dependent and should be evaluated empirically.

Summary. Pattern fixity does not enable precomputation because the memory contents depend entirely on unknown keys and the transcript structure binds all memory writes to the hash evolution. As a result, SCB is resistant to precomputation and amortization attacks.

6.3 Side-Channel and Data Independence

SCB uses data independent memory access. This is a core design property and was chosen to prevent timing and cache based side channels.

Data independent memory traversal. The scatter permutation π is computed solely from the buffer length L and fixed constants. It does not depend on the seed, the internal keys, or any sensitive data. Therefore the sequence of memory addresses accessed by the algorithm is identical for all inputs.

Constant time behavior of block generation. Each block has a fixed size and is always generated by a single call to SHAKE. No branching or data dependent control flow occurs during block generation. This ensures that execution time does not leak information about block contents.

Transcript absorption. The SHA3 transcript updates absorb fixed length encodings of indices and buffer slices. The frequency of full buffer absorption is also fixed and independent of sensitive data. This prevents timing variations based on secret dependent conditions.

Cache conditions. SCB intentionally induces cache misses by using a fixed stride larger than the L2 cache. While this increases runtime cost, it does so uniformly for all inputs. Thus the attacker cannot infer secret information by observing the timing of memory accesses or cache interactions.

Summary. The use of deterministic memory access, fixed size operations, and input-independent control flow removes the principal timing and cache-leakage channel associated with data-dependent memory-hard functions. This does not by itself prove resistance to all microarchitectural leakage, but it satisfies a key design requirement for password hashing and key derivation in side-channel-aware settings.

7 Performance and Implementation Considerations

The performance of SCB depends on three interacting factors: the memory cost parameter, the CPU cost parameter, and the latency properties of the target hardware. This section summarizes the scaling behavior of the construction, describes benchmark characteristics, and provides practical guidance for deployment and parameter selection.

7.1 Complexity and Scaling

The computational structure of SCB yields the following asymptotic cost characteristics.

Total work. The memory region has size

$$M = C_{\text{mem}} \cdot 2^{20} \text{ bytes,}$$

and there are $L = M/\text{CL}$ lines with $\text{CL} = 64$ bytes. Two distinct costs contribute to each iteration. First, generating and scattering all L lines costs $\Theta(L) = \Theta(C_{\text{mem}})$ work. Second, and decisively, the transcript absorbs the *entire* M -byte buffer once every $\text{LPR} = 4096$ writes; there are $L/\text{LPR} = 4 C_{\text{mem}}$ such snapshots per iteration, each hashing $M = C_{\text{mem}} \cdot 2^{20}$ bytes, for a snapshot cost of

$$4 C_{\text{mem}} \cdot C_{\text{mem}} \cdot 2^{20} = C_{\text{mem}}^2 \cdot 2^{22} \text{ bytes hashed per iteration.}$$

The total work therefore scales as

$$\Theta(C_{\text{cpu}} \cdot C_{\text{mem}}^2),$$

quadratic in the memory cost. This quadratic absorption term, not the linear fill term, dominates for all but the smallest C_{mem} , and it is the computational counterpart of the $\Theta(C_{\text{cpu}} L^2)$ transcript-work profile of Proposition 1: both originate in the periodic re-hashing of the full buffer.

Latency bound behavior. Since SCB uses a scatter pattern with a stride larger than the L2 cache size, each memory write causes an L2 miss and usually an L3 miss. This makes SCB latency dominated rather than bandwidth dominated. The effective time per memory block is determined by the latency of refilling a cache line from main memory rather than by the sustained memory bandwidth.

As a result, SCB scales differently from bandwidth focused MHFs such as Argon2. Two regimes should be distinguished. For small C_{mem} the linear fill term dominates and runtime is governed by cache-miss latency, so the cost per cache line written is roughly constant and largely insensitive to memory bandwidth. For larger C_{mem} the quadratic full-buffer

absorption term dominates, and runtime is governed by the throughput of the Keccak permutation over $\Theta(C_{\text{mem}}^2)$ bytes per iteration. The latency-resistance argument against GPUs is strongest in the former regime; in the latter, the cost is increasingly a compute (Keccak-throughput) cost rather than a pure latency cost, so GPU-resistance claims should be made with respect to the absorption workload as well as the scatter latency.

SHAKE and SHA3 costs. The cost of generating blocks through SHAKE is minor relative to the cost of memory accesses. The SHA3 transcript *index* absorptions and the per-iteration finalizations are likewise negligible. The periodic full-buffer absorption is the exception: it hashes the entire M -byte buffer $4C_{\text{mem}}$ times per iteration and constitutes the $\Theta(C_{\text{mem}}^2)$ term identified above. For small C_{mem} this is dominated by scatter latency and the profile is memory-latency bound; for large C_{mem} the absorption becomes the dominant cost and the profile is Keccak-throughput bound. Implementers tuning for a latency-bound profile should keep C_{mem} in the lower part of its range.

7.2 Benchmarks

This subsection summarizes general performance characteristics. Specific numerical results depend on processor architecture, memory hierarchy, and compiler optimizations, but several trends hold across platforms.

CPU performance. Measurements on common desktop processors (for example, recent Intel and AMD systems) show that SCB achieves its intended latency bound behavior. The cost per iteration is dominated by cache misses triggered by the scatter pattern. As CPU architectures improve in instruction throughput but not proportionally in memory latency, SCB maintains a roughly constant cost per block.

The effect of C_{cpu} is linear: doubling the CPU cost doubles total runtime. The effect of C_{mem} is super-linear once the full-buffer absorption dominates: because the absorbed work grows as $\Theta(C_{\text{mem}}^2)$, doubling C_{mem} can approach a fourfold increase in per-iteration cost at larger memory settings, while behaving closer to linearly at the smallest settings where cache-miss latency dominates.

Mobile and low power devices. On ARM based devices with smaller caches, SCB continues to produce cache thrashing behavior. The relative cost of memory misses is often higher on mobile hardware, which results in larger per iteration runtimes. For devices with limited memory, small values of C_{mem} may be necessary for acceptable performance.

GPU behavior. GPUs are optimized for throughput rather than serial memory latency. The scatter pattern induces irregular accesses across the full memory region, which reduces the ability of GPU warps to hide latency through rapid thread switching. As a result, SCB tends to perform worse on GPUs relative to CPUs, which is desirable for a memory hard function.

The expected behavior is that SCB narrows the gap between CPU and GPU cost compared with purely bandwidth-bound parameter sets, because the scatter traversal reduces locality and limits some forms of latency hiding. The actual CPU-to-GPU ratio is architecture-dependent and should be established with reproducible benchmarks before deployment claims are made.

Comparison with related MHFs. Compared with Argon2i, SCB produces fewer sequential memory reads but more cache evictions per iteration. This gives SCB stronger latency hardness but weaker sensitivity to memory bandwidth. Compared with scrypt, SCB avoids data dependent access and is therefore more resistant to cache timing attacks.

Compared with Balloon Hashing, SCB produces a more regular memory traversal pattern at L2 stride granularity, which simplifies implementation but retains similar cumulative memory complexity.

7.3 Engineering Guidance

We provide recommendations for selecting SCB parameters in practical deployments.

Parameter selection. For interactive authentication tasks, SCB should be configured so that a single evaluation takes between 50 ms and 250 ms on the target platform. This typically corresponds to values of:

- C_{mem} between 1 and 4,
- C_{cpu} between 1 and 2.

For server side key derivation with higher security requirements, larger values may be appropriate. The performance impact should be benchmarked on representative hardware.

Memory footprint. Since SCB allocates its full memory region at once, systems with limited physical memory may need to restrict C_{mem} to avoid paging. Paging would undermine the intended latency model and lead to unpredictable performance.

Avoiding parameter misuse. Very small values of C_{mem} (for example, $C_{\text{mem}} = 1$) reduce memory usage to the point where SCB becomes less effective against hardware acceleration. Implementations should default to a setting of at least $C_{\text{mem}} \geq 4$ for typical applications unless constrained by environment. If the L2 stride constant is retargeted, C_{mem} must additionally be chosen so that $L2 \mid M$ and the lane multiplier satisfies $\text{lmul} \geq 4$ (Section 2.3), so that the scatter map remains a permutation and the cache-miss guarantee is preserved.

Integration into KDF and password hashing flows. SCB can be used directly as a key derivation function or as the memory hard component within a larger construction such as HKDF or a password hashing envelope. When used for password hashing, SCB should be combined with a per user salt and an authenticated metadata field to avoid structural collisions. When used for key derivation, SCB can serve as the memory hard phase before a final context dependent derivation step.

Side channel safe implementation. Since SCB is data independent in its address schedule, its memory traversal avoids secret-dependent cache timing behavior. Implementations should ensure that SHAKE and SHA3 operations are constant time with respect to key material and that buffer clearing uses secure zeroization primitives.

Parallelism. SCB is designed to be sequential within each iteration. While iterations could be parallelized theoretically, doing so reduces security by weakening the dependency chain. Therefore production implementations should avoid parallelizing iterations and should preserve the sequential nature of block generation.

These considerations provide practical guidance for secure and efficient deployment of SCB across a wide range of platforms.

8 Comparison with Related Work

This section places SCB within the broader landscape of memory hard functions. We compare SCB with representative constructions from three major design families: matrix based MHFs such as Argon2, hashing based MHFs such as Balloon Hashing, and data dependent MHFs such as scrypt and yescrypt.

8.1 Comparison with Argon2 and Balloon Hashing

Argon2 and Balloon Hashing are both influential designs that shaped the expectations for secure and efficient MHFs. Although SCB shares some goals with these schemes, its structure and security properties differ in several respects.

Structure. Argon2 organizes memory into a two dimensional matrix. Each block depends on prior blocks selected according to a deterministic or data dependent indexing function. SCB uses a one dimensional buffer and a deterministic permutation of cache line indices derived from a fixed stride. The scatter based traversal forces memory access patterns that span the buffer at L2 sized intervals instead of relying on wide bandwidth use of contiguous memory segments.

Balloon Hashing repeatedly hashes the entire memory array in order to increase cumulative memory complexity. SCB incorporates a similar effect through periodic full buffer absorption into the transcript, although it does so conditionally at fixed stride boundaries rather than in every iteration of the hash.

Memory hardness. Argon2i derives hardness primarily from bandwidth saturation and from the difficulty of recomputing matrix blocks in time space tradeoff scenarios. SCB emphasizes latency hardness instead of bandwidth hardness. The cost of SCB is dominated by the high latency of sparse memory accesses rather than the number of bytes moved per unit time. This difference makes SCB less sensitive to improvements in memory bandwidth and more sensitive to architectural constraints on cache miss latency.

Balloon Hashing is designed to have provable cumulative memory complexity based on a hash based graph. SCB does not rely on repeated hashing of the entire memory region but binds the evolution of the internal key to sparse but strategically placed buffer snapshots. The resulting graph structure differs from the dense dependency pattern of Balloon Hashing, yet still maintains strong cumulative memory requirements.

Side channel properties. Argon2i uses data independent indexing to avoid cache timing leaks, while Argon2d uses data dependent indexing for additional hardness. SCB uses fully deterministic indexing similar to Argon2i. This makes SCB suitable for environments where side channel resistance is required.

Balloon Hashing uses data independent indexing as well, and SCB aligns with this approach by ensuring that the memory traversal pattern is fixed for each memory size, not for each input.

Hardware considerations. Argon2 achieves good performance on CPUs but can exhibit significant speedups on GPUs, especially for bandwidth heavy parameter sets. SCB attempts to shift the performance profile toward latency bound behavior. Since GPUs often struggle with irregular, low locality access patterns over large memory regions, SCB tends to reduce the relative hardware advantage of massively parallel devices.

8.2 Comparison with scrypt and yescrypt

Scrypt and yescrypt follow a different design philosophy from Argon2 and Balloon Hashing. They rely heavily on data dependent memory access, which provides strong resistance to certain attacks but introduces potential side channel issues. SCB differs from both in several key areas.

Data dependence. Scrypt uses data dependent memory lookup in its ROMix component. Yescrypt extends this idea with additional mixing rules. Although data dependence increases the cost of ASIC and GPU implementation, it can leak information about passwords or keys through timing channels or cache effects.

SCB avoids these concerns by using data independent access exclusively. All memory locations accessed by SCB depend only on public parameters such as memory size and cache stride. This makes SCB suitable for threat models that include timing or microarchitectural side channels.

Side channel resistance. Both scrypt and yescrypt require carefully implemented countermeasures to mitigate timing leaks caused by secret dependent memory access. SCB's deterministic access pattern provides an inherent protection against such channels. By design, SCB's runtime profile is identical for all seeds with fixed parameters.

Hardware resistance. Scrypt and yescrypt derive a significant portion of their hardness from the difficulty of implementing data dependent random memory access efficiently on custom hardware. SCB achieves hardware resistance through a different mechanism. The scatter pattern ensures that memory accesses occur at fixed intervals that exceed cache capacities. This forces repeated high latency operations that are difficult to pipeline or parallelize.

GPUs gain substantial advantage with scrypt only when they can hide latency through concurrent memory operations. The access pattern in SCB disperses memory requests in ways that limit the effectiveness of warp level scheduling. This results in CPU to GPU performance ratios that are higher than those obtained for scrypt under similar memory settings.

Cumulative memory cost. Yescrypt incorporates features that enable a wide variety of parameter tuning and supports large memory configurations efficiently. SCB uses a simpler one dimensional structure and a quadratic full-buffer transcript workload, $\Theta(C_{\text{cpu}}L^2)$ up to fixed stride constants for the honest evaluator (Proposition 1). This makes SCB straightforward to model while leaving tighter adversarial tradeoff bounds for future work.

Summary. SCB differs from scrypt and yescrypt primarily in its use of data independent access and cache latency driven hardness. These features make SCB more naturally aligned with side channel safe password hashing, while still maintaining significant resistance to hardware acceleration.

9 Limitations and Future Work

Although SCB meets the design goals of a latency bound memory hard function and provides a clear and well structured dependency model, several limitations and open questions remain. These limitations identify areas where further theoretical study or engineering refinement would strengthen confidence in the construction.

Tightness of the tradeoff curve. The present analysis establishes the sequential depth, honest peak-memory requirement, quadratic transcript-work profile, and reconstruction obligation faced by memory-reduced evaluators. What remains open is a tight lower bound for all streaming recomputation strategies across the full range of memory budgets, including models that jointly charge memory residence, Keccak permutation work, and replay of the keyed XOF stream. A treatment using refined pebbling or streaming-transcript models that pins down these constants is a natural direction for future work.

Fixed scatter pattern. SCB uses a deterministic permutation of cache line indices derived from the memory size and an assumed L2 cache size. This approach offers simplicity and avoids data dependent behavior, but it raises questions about whether a salt dependent or key dependent variation of the scatter permutation could strengthen security against specialized hardware. Investigating the tradeoff between fixed and keyed permutations and analyzing the impact on security and performance would refine the design.

Parameter tuning across architectures. The latency bound design of SCB depends on the memory hierarchy of the target device. While the chosen stride approximates typical L2 cache sizes on common architectures, performance characteristics differ across CPUs, mobile processors, and embedded systems. A systematic study of the optimal stride and memory layout for different architectures could improve both efficiency and security margins.

GPU and ASIC evaluation. Initial reasoning and small scale experiments suggest that SCB reduces the advantage of GPUs relative to CPUs. A more comprehensive evaluation that includes larger memory sizes, multiple GPU generations, and more realistic adversarial models would provide stronger empirical evidence for hardware resistance. Similarly, studying the behavior of SCB under ASIC oriented memory organizations could help quantify the cost of custom hardware implementations.

Integration with higher level protocols. Although SCB is suitable for password hashing and key derivation, integration into larger systems raises practical considerations. These include optimal combinations with salts, metadata encoding, password policy enforcement, and compatibility with existing hashing frameworks. Providing concrete guidance for these integration scenarios would support adoption in practice.

Formal verification. The SCB reference implementation is straightforward, but a formal verification of memory safety, constant time behavior, and compliance with the mathematical specification would enhance confidence in the design. The absence of data dependent branching makes SCB a good candidate for formal analysis and for integration into verified cryptographic libraries.

Comparison with alternative latency based MHFs. While SCB advances the study of latency bound memory hard functions, other designs may use different forms of address dispersion or hardware aware scheduling. Further exploration of this design space could clarify the relative strengths and weaknesses of SCB and help develop a richer theory of latency based hardness.

Summary. The SCB construction introduces a latency bound approach to memory hard design that is simple, deterministic, and compatible with side channel resistant implementations. Several theoretical and empirical questions remain open, and addressing these questions would strengthen the understanding of SCB and support its potential adoption in practical security systems.

10 Conclusion

SCB introduces a latency bound approach to memory hard function design that differs structurally from bandwidth oriented and data dependent constructions. By using a Keccak based initialization, a deterministic scatter based memory traversal, and a SHA3 based key evolution process, SCB induces a dependency graph with strong cumulative memory requirements while maintaining resistance to timing and cache based side channels. The engineering description presented in this paper provides a precise and implementation independent definition of the construction and states the implementation invariants required for analysis.

The graph based formalization captures the essential structural properties of SCB and supports a rigorous analysis of time space tradeoff resistance under the parallel pebbling model. We show that the full-buffer transcript induces sequential depth $\Omega(C_{\text{cpu}}L)$, honest peak memory $\Theta(M)$, and a quadratic transcript-work profile $\Theta(C_{\text{cpu}}L^2)$, while forcing memory-reduced evaluators to reconstruct omitted buffer state exactly. The sponge based security assumptions used for cSHAKE, SHAKE, and SHA3 align with standard analyses of Keccak and support the functional correctness and unpredictability properties required for secure key derivation.

The performance evaluation highlights the distinctive latency bound behavior of SCB. The construction scales linearly with the number of CPU iterations and quadratically with the memory cost parameter — the latter a direct consequence of the periodic full-buffer absorption that also underlies the transcript-reconstruction cost — and is designed to reduce the performance advantage of parallel hardware. This makes SCB a viable candidate for password hashing and memory hard key derivation on platforms where side channel resistance and hardware cost asymmetry are primary concerns.

Several open questions remain, particularly with regard to formal tradeoff bounds, hardware focused evaluation, and parameter optimization across device classes. These questions provide opportunities for further research and for strengthening the theoretical foundations of SCB. Overall, the design combines simplicity, deterministic behavior, and a clear memory hardness structure, making SCB a meaningful contribution to the ongoing study of memory hard function design.

References

1. Underhill, J. G. *SCB Specification*. Quantum Resistant Cryptographic Solutions Corporation, 2025. Available at: https://www.qrcscorp.ca/documents/scb_specification.pdf.
2. Underhill, J. G. *SCB Reference Implementation (C Source Code)*. Quantum Resistant Cryptographic Solutions Corporation, 2025. Available at: <https://github.com/QRC-S-CORP/QSC>.
3. Biryukov, A., Dinu, D., and Khovratovich, D. *Argon2: The Memory-Hard Function for Password Hashing and Other Applications*. PHC Final Report, 2015.
4. Boneh, D., Corrigan-Gibbs, H., and Schechter, S. *Balloon Hashing: A Memory-Hard Function Providing Provable Protection Against Sequential Attacks*. EuroS&P 2016.
5. Percival, C. *Stronger Key Derivation via Sequential Memory-Hard Functions*. scrypt Specification, 2009.
6. Bertoni, G., Daemen, J., Peeters, M., and Van Assche, G. *The Keccak Reference*. NIST SHA-3 Submission, 2011.
7. Bertoni, G., Daemen, J., Peeters, M., and Van Assche, G. *On the Indifferentiability of the Sponge Construction*. EUROCRYPT 2008, LNCS 4965, pp. 181–197.
8. Maurer, U., Renner, R., and Holenstein, C. *Indifferentiability, Impossibility Results on Reductions, and Applications to the Random Oracle Methodology*. TCC 2004, LNCS 2951, pp. 21–39.
9. Alwen, J., and Serbinenko, V. *High Parallel Complexity Graphs and Memory-Hard Functions*. STOC 2015, pp. 595–603.
10. Forler, C., Lucks, S., and Wenzel, J. *Catena: A Memory-Consuming Password-Scrambling Framework*. IACR ePrint 2013/525.
11. Peslyak, A. (Solar Designer). *yescrypt: Large-Scale Password Hashing*. Password Hashing Competition submission, 2014.
12. National Institute of Standards and Technology. *FIPS PUB 202: SHA-3 Standard — Permutation-Based Hash and Extendable-Output Functions*. 2015.
13. Kelsey, J., Change, S., and Perlner, R. *SP 800-185: SHA-3 Derived Functions — cSHAKE, KMAC, TupleHash, and ParallelHash*. NIST, 2016.